arXiv:1606.08415v5 [cs.LG] 6 Jun 2023

GAUSSIAN ERROR LINEAR UNITS (GELUS)

Dan Hendrycks*
University of California, Berkeley
hendrycks@berkeley.edu

Kevin Gimpel
Toyota Technological Institute at Chicago
kgimpel@ttic.edu

ABSTRACT

We propose the Gaussian Error Linear Unit (GELU), a high-performing neural network activation function. The GELU activation function is $x\Phi(x)$, where $\Phi(x)$ the standard Gaussian cumulative distribution function. The GELU nonlinearity weights inputs by their value, rather than gates inputs by their sign as in ReLUs ($x\mathbf{1}_{x>0}$). We perform an empirical evaluation of the GELU nonlinearity against the ReLU and ELU activations and find performance improvements across all considered computer vision, natural language processing, and speech tasks.

1 INTRODUCTION

Early artificial neurons utilized binary threshold units (Hopfield, 1982; McCulloch & Pitts, 1943). These hard binary decisions are smoothed with sigmoid activations, enabling a neuron to have a "firing rate" interpretation and to train with backpropagation. But as networks became deeper, training with sigmoid activations proved less effective than the non-smooth, less-probabilistic ReLU (Nair & Hinton, 2010) which makes hard gating decisions based upon an input's sign. Despite having less of a statistical motivation, the ReLU remains a competitive engineering solution which often enables faster and better convergence than sigmoids. Building on the successes of ReLUs, a recent modification called ELUs (Clevert et al., 2016) allows a ReLU-like nonlinearity to output negative values which sometimes increases training speed. In all, the activation choice has remained a necessary architecture decision for neural networks lest the network be a deep linear classifier.

Deep nonlinear classifiers can fit their data so well that network designers are often faced with the choice of including stochastic regularizer like adding noise to hidden layers or applying dropout (Srivastava et al., 2014), and this choice remains separate from the activation function. Some stochastic regularizers can make the network behave like an ensemble of networks, a pseudoensemble (Bachman et al., 2014), and can lead to marked accuracy increases. For example, the stochastic regularizer dropout creates a pseudoensemble by randomly altering some activation decisions through zero multiplication. Nonlinearities and dropout thus determine a neuron's output together, yet the two innovations have remained distinct. More, neither subsumed the other because popular stochastic regularizers act irrespectively of the input and nonlinearities are aided by such regularizers.

In this work, we introduce a new nonlinearity, the Gaussian Error Linear Unit (GELU). It relates to stochastic regularizers in that it is the expectation of a modification to Adaptive Dropout (Ba & Frey, 2013). This suggests a more probabilistic view of a neuron's output. We find that this novel nonlinearity matches or exceeds models with ReLUs or ELUs across tasks from computer vision, natural language processing, and automatic speech recognition.

2 GELU FORMULATION

We motivate our activation function by combining properties from dropout, zoneout, and ReLUs. First note that a ReLU and dropout both yield a neuron's output with the ReLU deterministically multiplying the input by zero or one and dropout stochastically multiplying by zero. Also, a new RNN regularizer called zoneout stochastically multiplies inputs by one (Krueger et al., 2016). We merge this functionality by multiplying the input by zero or one, but the values of this zero-one mask are stochastically determined while also dependent upon the input. Specifically, we can multiply the neuron input x by $m \sim \text{Bernoulli}(\Phi(x))$, where $\Phi(x) = P(X \leq$

*Work done while the author was at TTIC. Code available at github.com/hendrycks/GELUs

arXiv:1606.08415v5 [cs.LG] 6 Jun 2023

高斯误差线性单元（GELU函数）

丹·亨德里克斯*
加利福尼亚大学伯克利分校，邮箱：
hendrycks@berkeley.edu

凯文·金佩尔
芝加哥丰田技术研究院，邮箱：
kgimpel@ttic.edu

摘要

我们提出了高性能的神经网络激活函数——高斯误差线性单元（GELU）。GELU激活函数的表达式为 $x\Phi(x)$，其中 $\Phi(x)$即为标准的高斯累积分布函数。与ReLU函数不同，GELU这种非线性结构是根据输入值的大小来加权处理，而非像ReLU那样依据输入的正负符号来控制其传递与否($x\mathbf{1}_{x>0}$)。我们对GELU非线性结构与ReLU及ELU激活函数进行了实证对比测试，发现在所有涉及的计算机视觉、自然语言处理以及语音任务中，GELU的表现均更为优异。

1 引言

Early artificial neurons utilized binary threshold units (Hopfield, 1982; McCulloch & Pitts, 1943). These hard binary decisions are smoothed with sigmoid activations, enabling a neuron to have a "firing rate" interpretation and to train with backpropagation. But as networks became deeper, training with sigmoid activations proved less effective than the non-smooth, less-probabilistic ReLU (Nair & Hinton, 2010) which makes hard gating decisions based upon an input's sign. Despite having less of a statistical motivation, the ReLU remains a competitive engineering solution which often enables faster and better convergence than sigmoids. Building on the successes of ReLUs, a recent modification called ELUs (Clevert et al., 2016) allows a ReLU-like nonlinearity to output negative values which sometimes increases training speed. In all, the activation choice has remained a necessary architecture decision for neural networks lest the network be a deep linear classifier.

深度非线性分类器能够极其精准地拟合数据，因此网络设计者常常需要在为隐藏层添加噪声这类随机正则化方法与应用丢弃机制之间做出选择（Sri-Vastava等人，2014年），而这一选择与激活函数是相互独立的。某些随机正则化方法能让网络表现得像一个网络集成体，即伪集成（Bachman等人，2014年），进而显著提升准确率。例如，丢弃机制这种随机正则化方法通过零乘法随机改变某些激活函数的输出结果，从而形成伪集成。由此可见，非线性和丢弃机制共同决定了神经元的输出，但这两项技术至今仍保持着独立性。此外，它们也并未互相包含——因为常见的随机正则化方法在处理时并不考虑输入特征，而非线性特性也能借助这类正则化方法得到增强。

在本文中，我们引入了一种全新的非线性函数——高斯误差线性单元（GELU）。它与随机正则化器存在关联，因为其本质上是自适应丢弃机制（Ba & Frey, 2013）经过改进后的期望值。这一设计为神经元的输出提供了更具概率论意义的解释。实验结果表明，在计算机视觉、自然语言处理以及自动语音识别等各类任务中，这种新型非线性函数的性能均能与使用ReLU函数或ELU函数的模型相媲美，甚至更胜一筹。

2 GELU函数公式

我们通过结合丢弃机制、zoneout以及ReLU函数的特性来设计这种激活函数。首先需要注意的是，ReLU函数和丢弃机制都会让神经元的输出为特定值：ReLU函数以确定性方式将输入乘以零或一，而丢弃机制则以随机方式将输入乘以零。此外，还有一种名为zoneout的新的RNN正则化方法，它会以随机方式将输入乘以一（Krueger等人，2016年）。我们将这些功能整合在一起，即让输入乘以零或一，但这个零一掩码的具体数值是随机确定的，同时还会受到输入值的影响。具体而言，我们可以用 $m \sim \text{Bernoulli}(\Phi(x))$来将神经元输入 x 进行乘法运算，其中 $\Phi(x) = P(X \leq)$

*该成果为作者在TTIC工作期间完成，相关代码可于github.com/hendrycks/GELUs获取。

$x), X \sim \mathcal{N}(0,1)$ is the cumulative distribution function of the standard normal distribution. We choose this distribution since neuron inputs tend to follow a normal distribution, especially with Batch Normalization. In this setting, inputs have a higher probability of being "dropped" as x decreases, so the transformation applied to x is stochastic yet depends upon the input. Masking inputs in this fashion retains non-determinism but maintains dependency upon the input value. A stochastically chosen mask amounts to a stochastic zero or identity transformation of the input. This is much like Adaptive Dropout (Ba & Frey, 2013), but adaptive dropout is used in tandem with nonlinearities and uses a logistic not standard normal distribution. We found that it is possible to train competitive MNIST and TIMIT networks solely with this stochastic regularizer, all without using any nonlinearity.

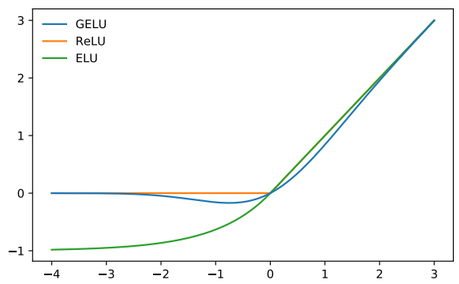


Figure 1: The GELU ($\mu = 0, \sigma = 1$), ReLU, and ELU ($\alpha = 1$).

We often want a deterministic decision from a neural network, and this gives rise to our new nonlinearity. The nonlinearity is the expected transformation of the stochastic regularizer on an input x, which is $\Phi(x) \times Ix + (1 - \Phi(x)) \times 0x = x\Phi(x)$. Loosely, this expression states that we scale x by how much greater it is than other inputs. Since the cumulative distribution function of a Gaussian is often computed with the error function, we define the Gaussian Error Linear Unit (GELU) as

$$\text{GELU}(x) = xP(X \leq x) = x\Phi(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}(x/\sqrt{2})\right].$$

We can approximate the GELU with

$$0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$$

or

$$x\sigma(1.702x),$$

if greater feedforward speed is worth the cost of exactness.

We could use different CDFs. For example we could use Logistic Distribution CDF $\sigma(x)$ to get what we call the Sigmoid Linear Unit (SiLU) $x\sigma(x)$. We could use the CDF of $\mathcal{N}(\mu, \sigma^2)$ and have μ and σ be learnable hyperparameters, but throughout this work we simply let $\mu = 0$ and $\sigma = 1$. Consequently, we do not introduce any new hyperparameters in the following experiments. In the next section, we show that the GELU exceeds ReLUs and ELUs across numerous tasks.

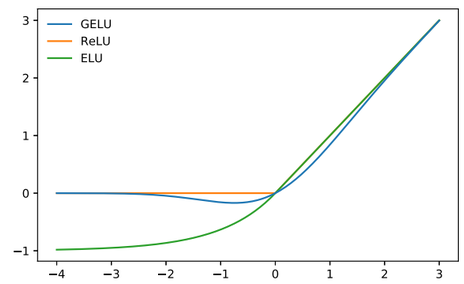
3 GELU Experiments

We evaluate the GELU, ELU, and ReLU on MNIST classification (grayscale images with 10 classes, 60k training examples and 10k test examples), MNIST autoencoding, Tweet part-of-speech tagging (1000 training, 327 validation, and 500 testing tweets), TIMIT frame recognition (3696 training, 1152 validation, and 192 test audio sentences), and CIFAR-10/100 classification (color images with 10/100 classes, 50k training and 10k test examples). We do not evaluate nonlinearities like the LReLU because of its similarity to ReLUs (see Maas et al. (2013) for a description of LReLUs).

3.1 MNIST Classification

Let us verify that this nonlinearity competes with previous activation functions by replicating an experiment from Clevert et al. (2016). To this end, we train a fully connected neural network with GELUs ($\mu = 0, \sigma = 1$), ReLUs, and ELUs ($\alpha = 1$). Each 8-layer, 128 neuron wide neural network is trained for 50 epochs with a batch size of 128. This experiment differs from those of

$x)$ $X \sim \mathcal{N}(0,1)$即为标准正态分布的累积分布函数。我们选择这种分布是因为神经元的输入往往遵循正态分布，尤其是在应用了批量归一化之后。在这种情形下，随着 x 的减小，输入被“丢弃”的概率会升高，因此施加在 x 上的变换虽具有随机性，但仍取决于输入值。以这种方式对输入进行遮蔽处理虽然仍保留了非确定性，但同时也保持了与输入值的依赖关系。通过随机方式选取的遮蔽矩阵实际上相当于对输入施加一种随机的零变换或恒等变换。这一机制与自适应丢弃机制（Ba & Frey, 2013）颇为相似，只不过自适应丢弃机制是与非线性函数结合使用的，并且采用逻辑斯蒂分布而非标准正态分布。我们发现，仅依靠这种随机正则化器，就完全可以训练出性能优异的MNIST和TIMIT模型，而且全程无需使用任何非线性函数。



我们通常希望从神经网络中获得确定的决策结果，而这便引出了我们所设计的新型非线性函数。该非线性函数即为预期的变换方式。

图1：GELU函数（$\mu = 0$，$\sigma = 1$）、ReLU函数以及ELU函数（$\alpha = 1$）。

随机正则化器对输入 x的作用，该输入为 $\Phi(x) \times Ix + (1 - \Phi(x)) \times 0x = x\Phi(x)$。简单来说，这一表达式表明我们会根据 x 相较于其他输入的数值大小差异来对其进行缩放。由于高斯分布的累积分布函数通常是通过误差函数来计算的，因此我们将高斯误差线性单元（GELU）定义为

$$\text{GELU}(x) = xP(X \leq x) = x\Phi(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}(x/\sqrt{2})\right].$$

我们可以用ELU函数来近似GELU函数

$$0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$$

or

$$x\sigma(1.702x),$$

如果更高的前馈速度值得为此付出精确度上的代价。

我们可以使用不同的累积分布函数。例如，我们可以利用逻辑斯蒂分布的累积分布函数 $\sigma(x)$来得到所谓的Sigmoid线性单元(SiLU) $x\sigma(x)$。我们也可以采用 $\mathcal{N}(\mu, \sigma^2)$的累积分布函数，并将$\mu$ 和 σ 设为可学习的超参数，但在本研究中我们直接让 $\mu = 0$ 和 $\sigma = 1$保持不变。因此，在后续实验中我们不会引入任何新的超参数。在下一节中，我们将证明GELU在众多任务中的表现均优于ReLU函数和ELU函数。

3项GELU函数实验

我们在MNIST分类任务（包含10个类别的灰度图像，共6万条训练样本和1万条测试样本）、MNIST自编码任务、推文词性标注任务（1000条训练推文、327条验证推文以及500条测试推文）、TIMIT音频帧识别任务（3696条训练音频、1152条验证音频以及192条测试音频句子），以及CIFAR-10/100分类任务（包含10/100个类别的彩色图像，分别有5万条训练样本和1万条测试样本）上，对GELU函数、ELU函数和ReLU函数进行了评估。由于LReLU函数与ReLU函数极为相似，我们并未对其这类非线性函数进行评估（关于LReLU的描述可参考Maas等人2013年的研究）。

3.1 MNIST数据集分类

让我们通过复现Clevert等人（2016年）的实验，来验证这种非线性结构是否真的能与以往的激活函数相竞争。为此，我们训练了一个包含GELU函数（$\mu = 0$，$\sigma = 1$）、ReLU函数以及ELU函数（$\alpha = 1$）的全连接神经网络。每个具有8层结构、128个神经元的神经网络均以128为批量大小，进行50个训练轮数的训练。该实验与之前的那些研究有所不同。

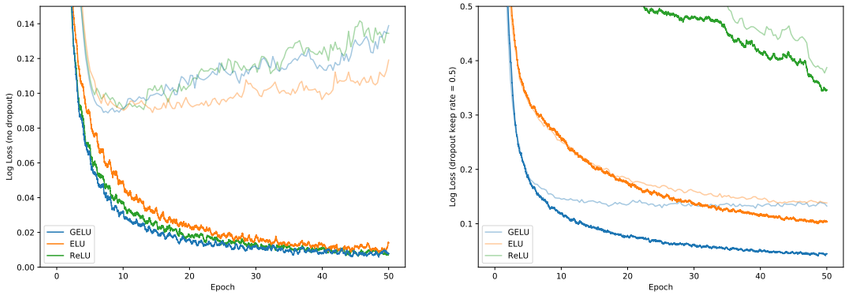


Figure 2: MNIST Classification Results. Left are the loss curves without dropout, and right are curves with a dropout rate of 0.5. Each curve is the the median of five runs. Training set log losses are the darker, lower curves, and the fainter, upper curves are the validation set log loss curves.

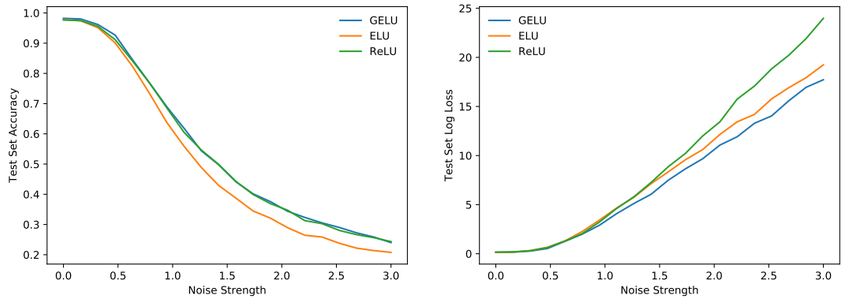


Figure 3: MNIST Robustness Results. Using different nonlinearities, we record the test set accuracy decline and log loss increase as inputs are noised. The MNIST classifier trained without dropout received inputs with uniform noise Unif$[-a, a]$ added to each example at different levels a, where $a = 3$ is the greatest noise strength. Here GELUs display robustness matching or exceeding ELUs and ReLUs.

Clevert et al. in that we use the Adam optimizer (Kingma & Ba, 2015) rather than stochastic gradient descent without momentum, and we also show how well nonlinearities cope with dropout. Weights are initialized with unit norm rows, as this has positive impact on each nonlinearity's performance (Hendrycks & Gimpel, 2016; Mishkin & Matas, 2016; Saxe et al., 2014). Note that we tune over the learning rates $\{10^{-3}, 10^{-4}, 10^{-5}\}$ with 5k validation examples from the training set and take the median results for five runs. Using these classifiers, we demonstrate in Figure 3 that classifiers using a GELU can be more robust to noised inputs. Figure 2 shows that the GELU tends to have the lowest median training log loss with and without dropout. Consequently, although the GELU is inspired by a different stochastic process, it comports well with dropout.

3.2 MNIST AUTOENCODER

We now consider a self-supervised setting and train a deep autoencoder on MNIST (Desjardins et al., 2015). To accomplish this, we use a network with layers of width 1000, 500, 250, 30, 250, 500, 1000, in that order. We again use the Adam optimizer and a batch size of 64. Our loss is the mean squared loss. We vary the learning rate from 10^{-3} to 10^{-4}. We also tried a learning rate of 0.01 but ELUs diverged, and GELUs and RELUs converged poorly. The results in Figure 4 indicate the GELU accommodates different learning rates and significantly outperforms the other nonlinearities.

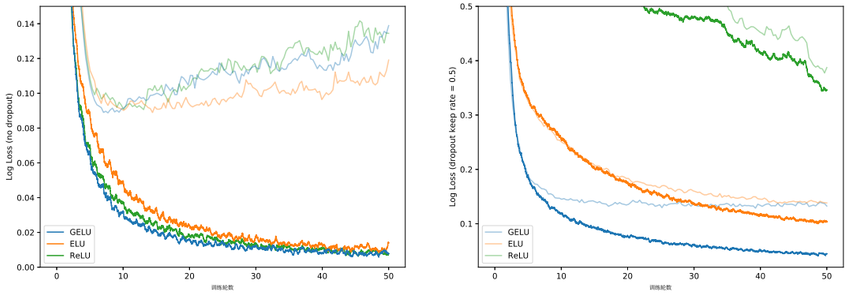


图2：MNIST分类结果。左侧为未应用丢弃机制时的损失曲线，右侧为应用了0.5比例丢弃机制后的曲线。每条曲线均为五次实验结果的中位数。颜色较深且位置较低的曲线代表训练集的对数损失，而颜色较浅且位置较高的曲线则代表验证集的对数损失曲线。

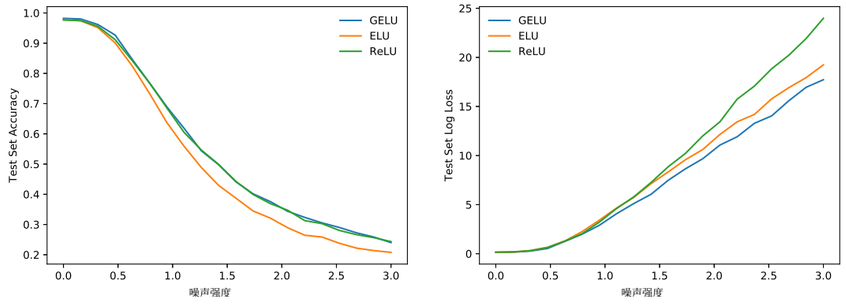


图3：MNIST鲁棒性测试结果。通过采用不同的非线性函数，我们记录了在输入数据加入噪声后测试集准确率的下降情况以及对数损失的上升趋势。未使用丢弃机制训练的MNIST分类器所接收的输入数据中，每个样本都会被添加不同强度的均匀噪声Unif$[-a, a]$，其中$a = 3$即为最大的噪声强度。实验结果表明，GELU函数的鲁棒性与ELU函数及ReLU函数相当甚至更优。

Clevert et al. in that we use the Adam optimizer (Kingma & Ba, 2015) rather than stochastic gradient descent without momentum, and we also show how well nonlinearities cope with dropout. Weights are initialized with unit norm rows, as this has positive impact on each nonlinearity's performance (Hendrycks & Gimpel, 2016; Mishkin & Matas, 2016; Saxe et al., 2014). Note that we tune over the learning rates $\{10^{-3}, 10^{-4}, 10^{-5}\}$ with 5k validation examples from the training set and take the median results for five runs. Using these classifiers, we demonstrate in Figure 3 that classifiers using a GELU can be more robust to noised inputs. Figure 2 shows that the GELU tends to have the lowest median training log loss with and without dropout. Consequently, although the GELU is inspired by a different stochastic process, it comports well with dropout.

3.2 MNIST自编码器

现在我们考虑一种自监督学习场景，并在MNIST数据集上训练一个深度自编码器（Desjardins等人，2015年）。为此，我们使用了层宽依次为1000、500、250、30、250、500、1000的网络结构。同样采用Adam优化器，批量大小设为64。我们的损失函数为均方误差损失。我们将学习率在 10^{-3} 到 10^{-4}之间进行调整。我们还尝试过0.01的学习率，但此时ELU函数出现发散现象，而GELU函数和ReLU函数的收敛效果也很差。图4中的结果表明，GELU函数能够适配不同的学习率，并且其性能显著优于其他非线性函数。

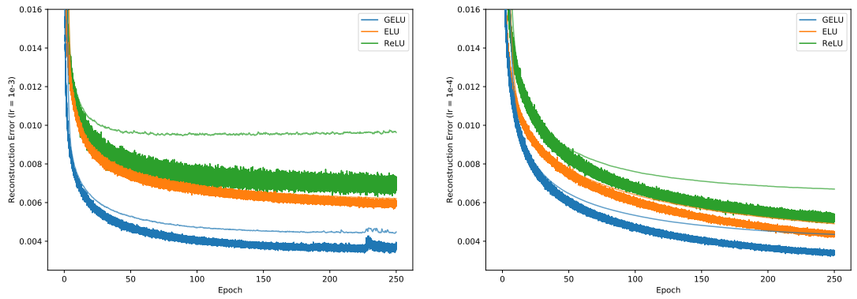


Figure 4: MNIST Autoencoding Results. Each curve is the median of three runs. Left are loss curves for a learning rate of 10^{-3}, and the right figure is for a 10^{-4} learning rate. Light, thin curves correspond to test set log losses.

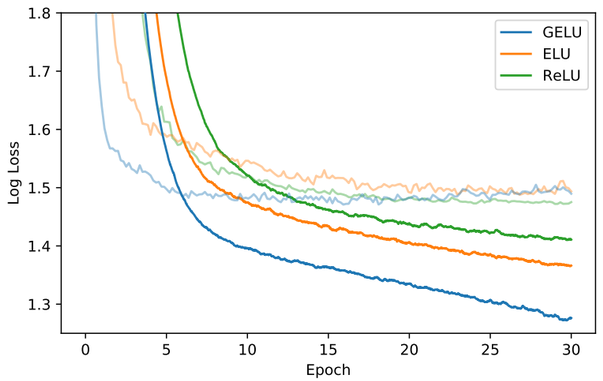


Figure 5: TIMIT Frame Classification. Learning curves show training set convergence, and the lighter curves show the validation set convergence.

3.3 TWITTER POS TAGGING

Many datasets in natural language processing are relatively small, so it is important that an activation generalize well from few examples. To meet this challenge we compare the nonlinearities on POS-annotated tweets (Gimpel et al., 2011; Owoputi et al., 2013) which contain 25 tags. The tweet tagger is simply a two-layer network with pretrained word vectors trained on a corpus of 56 million tweets (Owoputi et al., 2013). The input is the concatenation of the vector of the word to be tagged and those of its left and right neighboring words. Each layer has 256 neurons, a dropout keep probability of 0.8, and the network is optimized with Adam while tuning over the learning rates $\{10^{-3}, 10^{-4}, 10^{-5}\}$. We train each network five times per learning rate, and the median test set error is 12.57% for the GELU, 12.67% for the ReLU, and 12.91% for the ELU.

3.4 TIMIT FRAME CLASSIFICATION

Our next challenge is phone recognition with the TIMIT dataset which has recordings of 680 speakers in a noiseless environment. The system is a five-layer, 2048-neuron wide classifier as in (Mohamed et al., 2012) with 39 output phone labels and a dropout rate of 0.5 as in (Srivas-tava, 2013). This network takes as input 11 frames and must predict the phone of the center

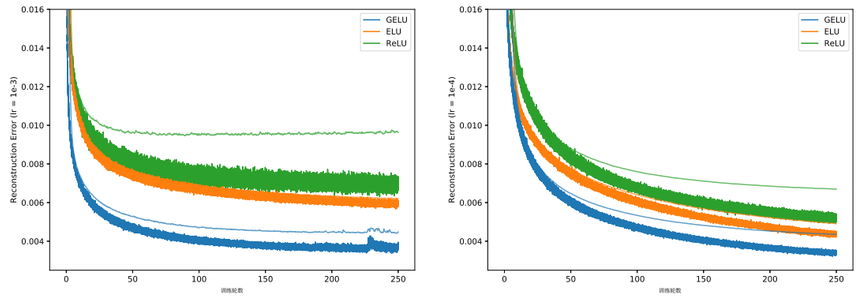


图4：MNIST自编码结果。每条曲线均为三次实验结果的中位数。左侧为学习率为 10^{-3}时的损失曲线，右侧则为学习率为 10^{-4} 时的损失曲线；较细的浅色曲线对应测试集的对数损失值。

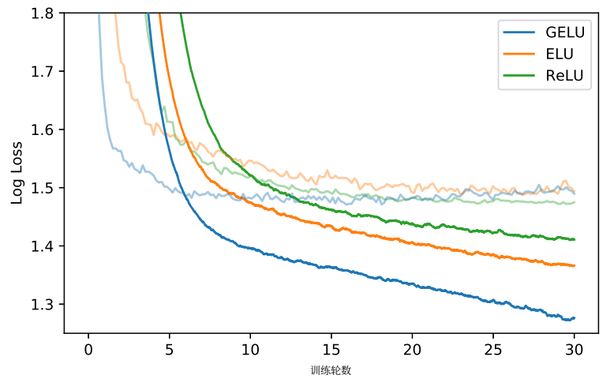


图5：TIMIT帧分类结果。学习曲线展示了训练集的收敛情况，而较浅的曲线则反映了验证集的收敛状况。

3.3 TWITTERPOS标签标注

Many datasets in natural language processing are relatively small, so it is important that an activation generalize well from few examples. To meet this challenge we compare the nonlinearities on POS-annotated tweets (Gimpel et al., 2011; Owoputi et al., 2013) which contain 25 tags. The tweet tagger is simply a two-layer network with pretrained word vectors trained on a corpus of 56 million tweets (Owoputi et al., 2013). The input is the concatenation of the vector of the word to be tagged and those of its left and right neighboring words. Each layer has 256 neurons, a dropout keep probability of 0.8, and the network is optimized with Adam while tuning over the learning rates $\{10^{-3}, 10^{-4}, 10^{-5}\}$. We train each network five times per learning rate, and the median test set error is 12.57% for the GELU, 12.67% for the ReLU, and 12.91% for the ELU.

3.4 TIMIT语音数据集帧分类

我们的下一个挑战是使用包含680位说话者录音、且处于无噪声环境中的TIMIT数据集进行语音识别。该系统采用与(Mohamed等人，2012)研究中相同的结构，即一个拥有2048个神经元、共五层的分类器，具备39个输出语音标签，并应用与(Srivas-tava，2013)研究一致的0.5的丢弃机制。该网络以11帧作为输入，需要预测其中位于中心位置的语音。

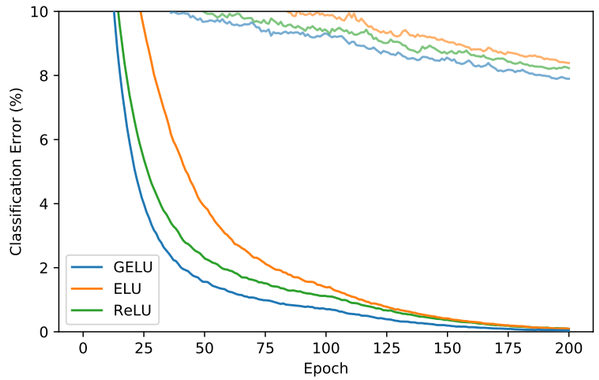


Figure 6: CIFAR-10 Results. Each curve is the median of three runs. Learning curves show training set error rates, and the lighter curves show the test set error rates.

frame using 26 MFCC, energy, and derivative features per frame. We tune over the learning rates $\{10^{-3}, 10^{-4}, 10^{-5}\}$ and optimize with Adam. After five runs per setting, we obtain the median curves in Figure 5, and median test error chosen at the lowest validation error is 29.3% for the GELU, 29.5% for the ReLU, and 29.6% for the ELU.

3.5 CIFAR-10/100 Classification

Next, we demonstrate that for more intricate architectures the GELU nonlinearity again outperforms other nonlinearities. We evaluate this activation function using CIFAR-10 and CIFAR-100 datasets (Krizhevsky, 2009) on shallow and deep convolutional neural networks, respectively.

Our shallower convolutional neural network is a 9-layer network with the architecture and training procedure from Salimans & Kingma (2016) while using batch normalization to speed up training. The architecture is described in appendix A and recently obtained state of the art on CIFAR-10 without data augmentation. No data augmentation was used to train this network. We tune over the learning initial rates $\{10^{-3}, 10^{-4}, 10^{-5}\}$ with 5k validation examples then train on the whole training set again based upon the learning rate from cross validation. The network is optimized with Adam for 200 epochs, and at the 100th epoch the learning rate linearly decays to zero. Results are shown in Figure 6, and each curve is a median of three runs. Ultimately, the GELU obtains a median error rate of **7.89**%, the ReLU obtains 8.16%, and the ELU obtains 8.41%.

Next we consider a wide residual network on CIFAR-100 with 40 layers and a widening factor of 4 (Zagoruyko & Komodakis, 2016). We train for 50 epochs with the learning rate schedule described in (Loshchilov & Hutter, 2016) ($T_0 = 50, \eta = 0.1$) with Nesterov momentum, and with a dropout keep probability of 0.7. Some have noted that ELUs have an exploding gradient with residual networks (Shah et al., 2016), and this is alleviated with batch normalization at the end of a residual block. Consequently, we use a Conv-Activation-Conv-Activation-BatchNorm block architecture to be charitable to ELUs. Over three runs we obtain the median convergence curves in Figure 7. Meanwhile, the GELU achieves a median error of **20.74**%, the ReLU obtains 21.77% (without our changes described above, the original 40-4 WideResNet with a ReLU obtains 22.89% (Zagoruyko & Komodakis, 2016)), and the ELU obtains 22.98%.

4 Discussion

Across several experiments, the GELU outperformed previous nonlinearities, but it bears semblance to the ReLU and ELU in other respects. For example, as $\sigma \to 0$ and if $\mu = 0$, the GELU becomes a ReLU. More, the ReLU and GELU are equal asymptotically. In fact, the GELU can be viewed as a way to smooth a ReLU. To see this, recall that ReLU $= \max(x, 0) = x\mathbb{1}(x > 0)$ (where

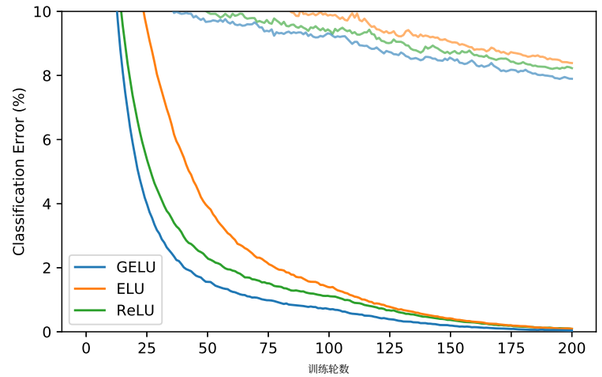


图6：CIFAR-10数据集的实验结果。每条曲线均为三次重复实验所得数据的中位数。学习曲线展示了训练集的错误率，而较浅色的曲线则表示测试集的错误率。

每帧会提取26个MFCC特征、能量特征以及对应的导数特征作为输入。我们会调整不同的学习率 $\{10^{-3}, 10^{-4}, 10^{-5}\}$，并使用Adam优化器进行优化。在每种设置下运行五次之后，我们得到了图5中所展示的中位数曲线；而在所有验证误差中最小的情况下，GELU函数对应的语音识别测试误差为29.3%，ReLU函数为29.5%，ELU函数则为29.6%。

3.5 CIFAR-10/100分类任务

接下来，我们将证明在结构更为复杂的模型中，GELU非线性函数依然能优于其他类型的非线性函数。我们分别利用CIFAR-10和CIFAR-100数据集（Krizhevsky于2009年的研究），在浅层和深层卷积神经网络上对这种激活函数进行了性能评估。

Our shallower convolutional neural network is a 9-layer network with the architecture and training procedure from Salimans & Kingma (2016) while using batch normalization to speed up training. The architecture is described in appendix A and recently obtained state of the art on CIFAR-10 without data augmentation. No data augmentation was used to train this network. We tune over the learning initial rates $\{10^{-3}, 10^{-4}, 10^{-5}\}$ with 5k validation examples then train on the whole training set again based upon the learning rate from cross validation. The network is optimized with Adam for 200 epochs, and at the 100th epoch the learning rate linearly decays to zero. Results are shown in Figure 6, and each curve is a median of three runs. Ultimately, the GELU obtains a median error rate of **7.89**%, the ReLU obtains 8.16%, and the ELU obtains 8.41%.

Next we consider a wide residual network on CIFAR-100 with 40 layers and a widening factor of 4 (Zagoruyko & Komodakis, 2016). We train for 50 epochs with the learning rate schedule described in (Loshchilov & Hutter, 2016) ($T_0 = 50, \eta = 0.1$) with Nesterov momentum, and with a dropout keep probability of 0.7. Some have noted that ELUs have an exploding gradient with residual networks (Shah et al., 2016), and this is alleviated with batch normalization at the end of a residual block. Consequently, we use a Conv-Activation-Conv-Activation-BatchNorm block architecture to be charitable to ELUs. Over three runs we obtain the median convergence curves in Figure 7. Meanwhile, the GELU achieves a median error of **20.74**%, the ReLU obtains 21.77% (without our changes described above, the original 40-4 WideResNet with a ReLU obtains 22.89% (Zagoruyko & Komodakis, 2016)), and the ELU obtains 22.98%.

4 讨论

在多项实验中，GELU函数的表现优于此前的各种非线性函数，但在其他方面却与ReLU函数和ELU函数有相似之处。例如，当 $\sigma \to 0$ 且 $\mu = 0$时，GELU函数就会转变为ReLU函数。此外，ReLU函数与GELU函数在渐近意义上也是相等的。实际上，GELU函数可以被视为对ReLU函数进行平滑处理的一种方式。要理解这一点，只需回想一下ReLU函数的表达式： $= \max(x,0) = x\mathbb{1}(x > 0)$（其中

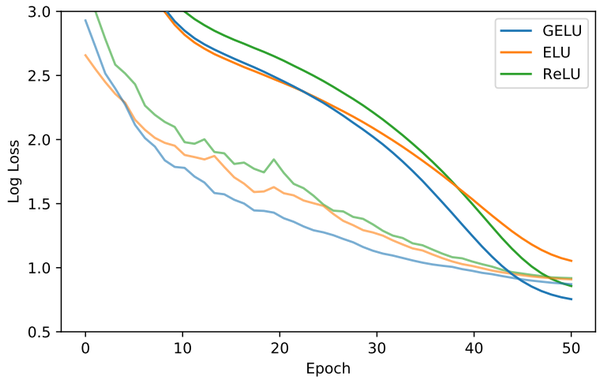


Figure 7: CIFAR-100 Wide Residual Network Results. Learning curves show training set convergence with dropout on, and the lighter curves show the test set convergence with dropout off.

$\mathbb{1}$ is the indicator function), while the GELU is $x\Phi(x)$ if $\mu = 0, \sigma = 1$. Then the CDF is a smooth approximation to the binary function the ReLU uses, like how the sigmoid smoothed binary threshold activations. Unlike the ReLU, the GELU and ELU can be both negative and positive. In fact, if we used the cumulative distribution function of the standard Cauchy distribution, then the ELU (when $\alpha = 1/\pi$) is asymptotically equal to $xP(C \leq x), C \sim \text{Cauchy}(0, 1)$ for negative values and for positive values is $xP(C \leq x)$ if we shift the line down by $1/\pi$. These are some fundamental relations to previous nonlinearities.

However, the GELU has several notable differences. This non-convex, non-monotonic function is not linear in the positive domain and exhibits curvature at all points. Meanwhile ReLUs and ELUs, which are convex and monotonic activations, are linear in the positive domain and thereby can lack curvature. As such, increased curvature and non-monotonicity may allow GELUs to more easily approximate complicated functions than can ReLUs or ELUs. Also, since $\text{ReLU}(x) = x\mathbb{1}(x > 0)$ and $\text{GELU}(x) = x\Phi(x)$ if $\mu = 0, \sigma = 1$, we can see that the ReLU gates the input depending upon its sign, while the GELU weights its input depending upon how much greater it is than other inputs. In addition and significantly, the GELU has a probabilistic interpretation given that it is the expectation of a stochastic regularizer.

We also have two practical tips for using the GELU. First we advise using an optimizer with momentum when training with a GELU, as is standard for deep neural networks. Second, using a close approximation to the cumulative distribution function of a Gaussian distribution is important. A sigmoid function $\sigma(x) = 1/(1 + e^{-x})$ is an approximation of a cumulative distribution function of a normal distribution. However, we found that a Sigmoid Linear Unit (SiLU) $x\sigma(x)$ performs worse than GELUs but usually better than ReLUs and ELUs, so our SiLU is also a reasonable nonlinearity choice. Instead of using a $x\sigma(x)$ to approximate $\Phi(x)$, we used $0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$ (Choudhury, 2014)[1] or $x\sigma(1.702x)$. Both are sufficiently fast, easy-to-implement approximations, and we used the former in every experiment in this paper.

5 Conclusion

For the numerous datasets evaluated in this paper, the GELU exceeded the accuracy of the ELU and ReLU consistently, making it a viable alternative to previous nonlinearities.

[1]Thank you to Dmytro Mishkin for bringing an approximation like this to our attention.

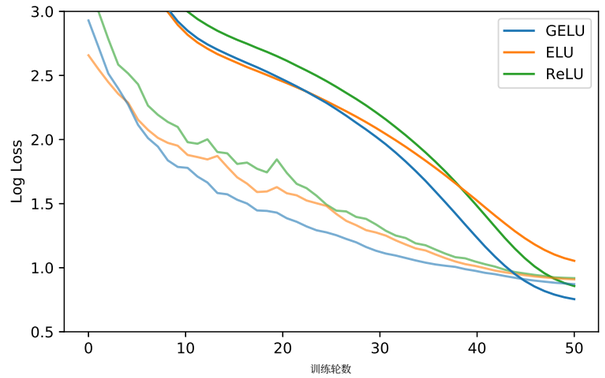


图7：CIFAR-100数据集下的宽残差网络实验结果。学习曲线展示了开启丢弃机制时的训练集收敛情况，而较浅的曲线则显示了关闭丢弃机制时的测试集收敛情况。

1即为指示函数），而当 $\mu = 0, \sigma = 1$时，GELU的表达式为 $x\Phi(x)$。此时，累积分布函数便是ReLU所使用的二元函数的平滑近似形式，就如同Sigmoid函数对二元阈值激活函数进行了平滑处理一样。与ReLU不同，GELU和ELU的输出值既可以是负数也可以是正数。实际上，如果我们使用标准柯西分布的累积分布函数，那么当 $\alpha = 1/\pi$时，ELU对于负数值而言渐近等于 $xP(C \leq x)$ $C \sim \text{Cauchy}(0, 1)$，而对于正数值而言，若将函数曲线向下平移 $1/\pi$个单位，则其表达式为 $xP(C \leq x)$。这些就是与以往各种非线性函数相关的若干基本关系。

不过，GELU函数存在几处显著的差异。这一非凸且非单调的函数在正数域内并非线性，且在所有点上都具有曲率。而作为凸函数且单调的激活函数，ReLU函数和ELU函数在正数域内是线性的，因此可能不存在曲率。正因如此，更高的曲率与非单调性使得GELU函数比ReLU函数或ELU函数更能轻松逼近复杂的函数。此外，由于当 $\mu = 0, \sigma = 1$时，ReLU函数表现为$\text{ReLU}(x) = x\mathbb{1}(x > 0)$，GELU函数则表现为$\text{GELU}(x) = x\Phi(x)$，我们可以看出ReLU函数会根据输入的符号来决定是否通过该输入，而GELU函数则会根据输入相较于其他输入的超出程度来为其赋予权重。更值得注意的是，由于GELU函数是一个随机正则化器的期望值，因此它还具有概率意义上的解释。

关于如何使用GELU函数，我们还有两个实用建议。首先，与深度神经网络的常规做法一致，建议在采用GELU进行训练时使用带有动量的优化器。其次，使用高斯分布的累积分布函数的近似值非常重要。Sigmoid函数 $\sigma(x) = 1/(1 + e^{-x})$其实就是正态分布累积分布函数的一种近似形式。不过我们发现，虽然Sigmoid线性单元（SiLU）$x\sigma(x)$的性能不如GELU函数，但通常优于ReLU函数和ELU函数，因此它也是一种可行的非线性函数选择。我们没有选用 $x\sigma(x)$来近似 $\Phi(x)$，而是采用了$0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$（Choudhury, 2014）[1]或 $x\sigma(1.702x)$这两种方式。这两种方法都具备足够的运算速度且易于实现，而在本文的所有实验中我们都选择了前者。

5 结论

在本文评估的众多数据集中，GELU函数的性能始终优于ELU函数和ReLU函数，在精度方面表现更佳，因此可作为传统非线性函数的优质替代方案。

[1]感谢 Dmytro Mishkin 将这样的近似方法介绍给我们。

ACKNOWLEDGMENT

We would like to thank NVIDIA Corporation for donating several TITAN X GPUs used in this research.

REFERENCES

Jimmy Ba and Brendan Frey. Adaptive dropout for training deep neural networks. In *Neural Information Processing Systems*, 2013.

Philip Bachman, Ouais Alsharif, and Doina Precup. Learning with pseudo-ensembles. In *Neural Information Processing Systems*, 2014.

Amit Choudhury. A simple approximation to the area under standard normal curve. In *Mathematics and Statistics*, 2014.

Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). In *International Conference on Learning Representations*, 2016.

Guillaume Desjardins, Karen Simonyan, Razvan Pascanu, and Koray Kavukcuoglu. Natural neural networks. In *arXiv*, 2015.

Kevin Gimpel, Nathan Schneider, Brendan O′Connor, Dipanjan Das, Daniel Mills, Jacob Eisenstein, Michael Heilman, Dani Yogatama, Jeffrey Flanigan, and Noah A. Smith. *Part-of-Speech Tagging for Twitter: Annotation, Features, and Experiments*. Association for Computational Linguistics (ACL), 2011.

Dan Hendrycks and Kevin Gimpel. Adjusting for dropout variance in batch normalization and weight initialization. In *arXiv*, 2016.

John Hopfield. Neural networks and physical systems with emergent collective computational abilities. In *Proceedings of the National Academy of Sciences of the USA*, 1982.

Diederik Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. International Conference for Learning Representations, 2015.

Alex Krizhevsky. Learning Multiple Layers of Features from Tiny Images, 2009.

David Krueger, Tegan Maharaj, János Kramár, Mohammad Pezeshki, Nicolas Ballas, Nan Rosemary Ke1, Anirudh Goyal, Yoshua Bengio, Hugo Larochelle, Aaron Courville, and Chris Pal. Zoneout: Regularizing RNNs by randomly preserving hidden activations. In *Neural Information Processing Systems*, 2016.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with restarts. *arXiv*, 2016.

Andrew L. Maas, Awni Y. Hannun, , and Andrew Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *International Conference on Machine Learning*, 2013.

Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. In *Bulletin of Mathematical Biophysics*, 1943.

Dmytro Mishkin and Jiri Matas. All you need is a good init. In *International Conference on Learning Representations*, 2016.

Abdelrahman Mohamed, George E. Dahl, and Geoffrey E. Hinton. Acoustic modeling using deep belief networks. In *IEEE Transactions on Audio, Speech, and Language Processing*, 2012.

Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *International Conference on Machine Learning*, 2010.

Olutobi Owoputi, Brendan O'Connor, Chris Dyer, Kevin Gimpel, Nathan Schneider, and Noah A. Smith. Improved part-of-speech tagging for online conversational text with word clusters. In *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2013.

致谢

在此，我们要感谢NVIDIA公司捐赠的多块TITAN X GPU，这些硬件为本次研究提供了重要支持。

参考文献

Jimmy Ba与Brendan Frey所著《用于训练深度神经网络的自适应丢弃机制》，发表于*Neural Information Processing Systems*，2013年。

Philip Bachman、Ouais Alsharif和Doina Precup所著《基于伪集成算法的学习方法》，发表于*NeuralInformationProcessingSystems*，2014年。

Amit Choudhury所著《标准正态曲线下面积的简易近似方法》，发表于*Mathematicsand Statistics*，2014年。

Djork-Arn´e Clevert、Thomas Unterthiner和Sepp Hochreiter所著《利用指数线性单元实现快速且精确的深度网络学习》，发表于*International Conference on Learning Representations*，2016年。

Guillaume Desjardins、Karen Simonyan、Razvan Pascanu与Koray Kavukcuoglu。自然神经网络。载于*arXiv*，2015年。

凯文·金佩尔、内森·施奈德、布伦丹·O′康纳、迪潘詹·达斯、丹尼尔·米尔斯、雅各布·艾森斯坦、迈克尔·海尔曼、达尼·约加塔马、杰弗里·弗拉尼根以及诺亚·A·史密斯。《*Twitter*语料的分词标注：标注方法、特征与实验》。计算语言学协会（ACL），2011年。

丹·亨德里克斯与凯文·金佩尔。针对批量归一化及权重初始化中的丢弃机制方差进行校正。发表于*arXiv*，2016年。

约翰·霍普菲尔德。《具有涌现性集体计算能力的神经网络与物理系统》。载于《美国国家科学院院刊》，1982年。

Diederik Kingma与Jimmy Ba。《*Adam*：一种用于随机优化的方法》。2015年国际学习表征会议。

Alex Krizhevsky。《从极小尺寸图像中学习多层特征》，2009年。

DavidKrueger、Tegan Maharaj、J´anos Kram´ar、Mohammad Pezeshki、Nicolas Ballas、Nan Rosemary Ke1、Anirudh Goyal、Yoshua Bengio、Hugo Larochelle、Aaron Courville以及Chris Pal。Zoneout：通过随机保留隐藏层的激活函数来实现RNN的正则化。载于神经信息处理系统会议，2016年。

IlYa Loshchilov与Frank Hutter所著的《SGDR：带重启机制的随机梯度下降法》。*arXiv*，201年 6.

Andrew L. Maas、Awni Y. Hannun以及Andrew Y. Ng。整流器非线性特性可提升神经网络声学模型性能。载于国际机器学习大会，2013年。

沃伦·S·麦卡洛克与沃尔特·皮茨。《神经活动中内在思想的逻辑演算》。载于《数学生物物理学公报》，1943年。

Dmytro Mishkin与Jiri Matas指出，只要选用合适的初始参数即可。该文发表于学习表征国际会议，时间为2016年。

Abdelrahman Mohamed、George E. Dahl以及Geoffrey E. Hinton研究了基于深度信念网络的声学建模方法，相关论文发表在*IEEE*音频、语音与语言处理交易刊，时间为2012年。

Vinod Nair与Geoffrey E. Hinton证明，修正线性单元能够提升受限玻尔兹曼机的性能，这篇研究结果发表在机器学习国际会议，时间为2010年。

奥卢托比·奥沃普蒂、布伦丹·奥康纳、克里斯·戴尔、凯文·金佩尔、内森·施奈德以及诺亚·A·史密斯。基于词群分析的在线对话文本词性标注改进方法。《计算语言学协会北美分会（*NAACL*）》，2013年。

Tim Salimans and Diederik P. Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *Neural Information Processing Systems*, 2016.

Andrew M. Saxe, James L. McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations*, 2014.

Anish Shah, Sameer Shinde, Eashan Kadam, Hena Shah, and Sandip Shingade. Deep residual networks with exponential linear unit. In *Vision Net*, 2016.

Nitish Srivastava. Improving neural networks with dropout. In *University of Toronto*, 2013.

Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. In *Journal of Machine Learning Research*, 2014.

Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. *British Machine Vision Conference*, 2016.

Tim Salimans and Diederik P. Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *Neural Information Processing Systems*, 2016.

Andrew M. Saxe, James L. McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations*, 2014.

Anish Shah, Sameer Shinde, Eashan Kadam, Hena Shah, and Sandip Shingade. Deep residual networks with exponential linear unit. In *Vision Net*, 2016.

Nitish Srivastava. Improving neural networks with dropout. In *University of Toronto*, 2013.

Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. In *Journal of Machine Learning Research*, 2014.

Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. *British Machine Vision Conference*, 2016.

8

A NEURAL NETWORK ARCHITECTURE FOR CIFAR-10 EXPERIMENTS

Table 1: Neural network architecture for CIFAR-10.

Layer Type	**# channels**	x, y **dimension**
raw RGB input	3	32
ZCA whitening	3	32
Gaussian noise $\sigma = 0.15$	3	32
3×3 conv with activation	96	32
3×3 conv with activation	96	32
3×3 conv with activation	96	32
2×2 max pool, stride 2	96	16
dropout with $p = 0.5$	96	16
3×3 conv with activation	192	16
3×3 conv with activation	192	16
3×3 conv with activation	192	16
2×2 max pool, stride 2	192	8
dropout with $p = 0.5$	192	8
3×3 conv with activation	192	6
1×1 conv with activation	192	6
1×1 conv with activation	192	6
global average pool	192	1
softmax output	10	1

B HISTORY OF THE GELU AND SILU

This paper arose from DH's first research internship as an undergraduate in June 2016. The start of the week after, this paper was put on arXiv, in which we discuss smoother ReLU activation functions ($x \times P(X \leq x)$) and their relation to stochastic regularizers. In 2016, we submitted the paper to ICLR and made the paper and code publicly available. In the paper, we introduced and coined the Sigmoid Linear Unit (SiLU) as $x \cdot \sigma(x)$.

In the first half of 2017, Elfwing et al. published a paper that proposed the same activation function as SiLU, $x \cdot \sigma(x)$, which they called "SIL." At the end of 2017, over a year after this paper was first released, Quoc Le and others from Google Brain put out a paper proposing $x \cdot \sigma(x)$ without citing either the Elfwing et al. paper or this work. Upon learning this, we contacted both parties. Elfwing quickly updated their work to call the activation the "SiLU" instead of "SIL" to recognize that we originally introduced the activation.

Unlike Elfwing et al., the Google Brain researchers continued calling the activation "swish." However, there was no novelty. The first author of the "swish" paper stated their oversight in public, saying, "As has been pointed out, we missed prior works that proposed the same activation function. The fault lies entirely with me for not conducting a thorough enough literature search." To subdue criticism, an update to the paper was released a week later. Rather than give credit to this work for the SiLU, the update only cited this work for the GELU so that the "swish" appeared more novel. In the updated paper, a learnable hyperparameter β was introduced, and the swish was changed from $x \cdot \sigma(x)$ to $x \cdot \sigma(\beta \cdot x)$. This staked all of the idea's novelty on an added learnable hyperparameter β.

Despite the addition of the hyperparameter beta, nearly all of the community still used the original "swish" function without β (i.e., with $\beta = 1$). Since this paper was from Google Brain, the Tensorflow implementation ended up being called "swish," and the default setting removed β, rendering it identical to the SiLU. The practice of adding an unused hyperparameter allowed claiming novelty while effectively receiving credit for an idea that originated elsewhere. Future papers with the same senior authors persistently referred to the "swish" function even when not using β, making it identical to the SiLU, originally proposed in this work. This resulted in the "swish" paper inappropriately gaining credit for the idea.

用于CIFAR-10实验的神经网络架构

表1：CIFAR-10对应的神经网络架构。

层类型	**通道数**	x， y **维度**
原始RGB输入	3	32
ZCA标准化算法白化处理	3	32
高斯噪声 $\sigma = 0.15$	3	32
3×3 带有激活函数的卷积操作	96	32
3×3 带有激活函数的卷积操作	96	32
3×3 带有激活函数的卷积操作	96	32
2×2 最大值池化，步长为2	96	16
带有 $p = 0.5$的丢弃机制	96	16
3×3 带有激活函数的卷积操作	192	16
3×3 带有激活函数的卷积操作	192	16
3×3 带有激活函数的卷积操作	192	16
2×2 最大值池化，步长为2	192	8
带有 $p = 0.5$的丢弃机制	192	8
3×3 带有激活函数的卷积操作	192	6
1×1 带有激活函数的卷积操作	192	6
1×1 带有激活函数的卷积操作	192	6
全局平均池化	192	1
softmax输出	10	1

B GELU函数与SiLU激活函数的发展历史

这篇论文源于DH在2016年6月作为本科生进行的第一次研究实习。一周后，该论文被上传至arXiv平台，在文中我们探讨了更为平滑的ReLU激活函数（$x \times P(X \leq x)$）以及它们与随机正则化器之间的关系。2016年，我们将这篇论文提交给了国际机器学习大会（ICLR），并公开发布了对应的论文及代码。在论文中，我们首次提出了Sigmoid线性单元（SiLU），并将其定义为 $x \cdot \sigma(x)$。

2017年上半年，Elfwing等人发表了一篇论文，提出了与SiLU相同的 $x \cdot \sigma(x)$激活函数，并将其命名为“SIL”。在这篇论文首次发布一年多后的2017年底，来自Google Brain的Quoc Le等人又发表了一篇论文，提出了 $x \cdot \sigma(x)$，却既没有引用Elfwing等人的那篇论文，也没有提及这项工作。得知这一情况后，我们联系了双方。Elfwing很快修正了他们的研究，将该激活函数重新命名为“SiLU”而非“SIL”，以此承认是我们率先提出了这一激活函数。

与Elfwing等人不同，Google Brain的研究人员仍继续将这种激活函数称为‘swish’。然而这其实并无新意。那篇介绍‘swish’的论文的第一作者公开承认了自己的疏忽，他表示：‘正如有人指出的那样，我们忽略了此前已有研究提出了相同的激活函数。未能进行足够全面的文献检索，过错完全在我。’为了平息批评，他们在一周后发布了该论文的修正版。修正版并未将此项工作归功于SiLU激活函数，而只是将其与GELU函数联系起来，以此让‘swish’看起来更具创新性。在修正后的论文中，研究人员引入了一个可学习的超参数 β，同时将swish函数的表达式从$x \cdot \sigma(x)$改为 $x \cdot \sigma(\beta \cdot x)$。这样一来，整个想法的创新性就完全依赖于这个新增的可学习超参数 β了。

尽管加入了超参数beta，但几乎整个学术界仍然沿用原有的‘swish’函数，而没有使用 β（即仍采用 $\beta = 1$）。由于这篇论文出自Google Brain，TensorFlow中的实现也被命名为‘swish’，而且其默认设置去掉了 β，这使得该函数实际上与SiLU函数完全相同。通过添加一个并未实际使用的超参数，相关研究者既能宣称自己的创新性，又能将原本属于其他研究的成果据为己有。后续由这些同一批资深作者撰写的论文，即便在并不使用 β的情况下，依然持续将函数称为‘swish’，从而使其与最初在这项研究中提出的SiLU函数混为一谈，最终导致那篇关于‘swish’的论文不恰当地获得了该思想的荣誉。

Things changed as the GELU began to be used in BERT and GPT, becoming the default activation for state-of-the-art Transformers. Now it is substantially more commonly used than the SiLU.

Separately, a reddit post “Google has a credit assignment problem in research” became popular and focused on how they refer to the SiLU as the swish. As an example, they mentioned “Smooth Adversarial Training” as an example of poor credit assignment. In the “Smooth Adversarial Training” paper, which came from the senior author of the swish, the term “swish” was used instead of “SiLU.” To reduce blowback from the post, the authors updated the paper and replaced “swish” with the “SiLU,” recognizing this paper as the original source of the idea. After this post, popular libraries such as Tensorflow and PyTorch also began to rename the function to “SiLU” instead of “swish.” For close observers, this issue has been largely settled, and we are grateful for the proper recognition that has largely come to pass.

随着GELU函数开始被应用于BERT和GPT模型，并成为最先进的Transformer模型中的默认激活函数，情况发生了变化。如今，它的使用频率已经远远超过了SiLU函数。

另外，一篇发布在reddit平台上的帖子《谷歌公司在研究中面临信用分配问题》广为流传，该帖子重点探讨了为何人们会将SiLU激活函数称为swish。例如，帖子中提到“平滑对抗训练”就是信用分配不当的典型案例。在那篇由swish激活函数的资深提出者撰写的“平滑对抗训练”论文中，实际上使用的是“swish”这一名称而非“SiLU”。为了平息该帖子引发的争议，论文作者对文章进行了修改，将“swish”替换回“SiLU”，并承认该论文才是这一概念的最初来源。在这篇帖子出现之后，Tensorflow和PyTorch等主流机器学习库也相继开始将该函数从“swish”改名为“SiLU”。对于密切关注这一领域进展的人士而言，这一问题现已基本得到解决，我们也为这一概念最终得到了应有的认可而感到欣慰。

10